Pandas is defined as an open-source library that provides high-performance data manipulation in Python. It is used for data analysis in Python and developed by Wes McKinney in 2008.

Pandas Introduction:

Pandas is defined as an open-source library that provides high-performance data manipulation in Python. The name of Pandas is derived from the word Panel Data, which means an Econometrics from Multidimensional data.

Data analysis requires lots of processing, such as restructuring, cleaning or merging, etc. There are different tools are available for fast data processing, such as Numpy, Scipy, Cython, and Panda. But we prefer Pandas because working with Pandas is fast, simple and more expressive than other tools.

Pandas is built on top of the Numpy package, means Numpy is required for operating the Pandas. Before Pandas, Python was capable for data preparation, but it only provided limited support for data analysis. So, Pandas came into the picture and enhanced the capabilities of data analysis. It can perform five significant steps required for processing and analysis of data irrespective of the origin of the data, i.e., load, manipulate, prepare, model, and analyze.

Key Features of Pandas:

It has a fast and efficient DataFrame object with the default and customized indexing. Used for reshaping and pivoting of the data sets. Group by data for aggregations and transformations. It is used for data alignment and integration of the missing data. Provide the functionality of Time Series. Process a variety of data sets in different formats like matrix data, tabular heterogeneous, time series. Handle multiple operations of the data sets such

as subsetting, slicing, filtering, groupBy, re-ordering, and re-shaping. It integrates with the other libraries such as SciPy, and scikit-learn.

Benefits of Pandas

The benefits of pandas over using other language are as follows:

Data Representation: It represents the data in a form that is suited for data analysis through its DataFrame and Series.

Clear code: The clear API of the Pandas allows you to focus on the core part of the code. So, it provides clear and concise code for the user. Python Pandas Data Structure

The Pandas provides two data structures for processing the data, i.e., Series and DataFrame, which are discussed below:

1) Series

It is defined as a one-dimensional array that is capable of storing various data types. The row labels of series are called the index. We can easily convert the list, tuple, and dictionary into series using "series' method. A Series cannot contain multiple columns. Data: It can be any list, dictionary, or scalar value.

2) DataFrame

It is a widely used data structure of pandas and works with a two-dimensional array with labeled axes (rows and columns). DataFrame is defined as a standard way to store data and has two different indexes, i.e., row index and column index. It consists of the following properties:

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kalathil, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778121213

The columns can be heterogeneous types like int, bool, and so on. It can be seen as a dictionary of Series structure where both the rows and columns are indexed. It is denoted as **"columns"** in case of columns and **"index"** in case of rows.

```
pandas as pd
```

Pandas is usually imported under the pd alias.

alias: In Python alias are an alternate name for referring to the same thing.

Create an alias with the as keyword while importing:

```
import pandas as pd
```

Checking Pandas Version

The version string is stored under **__version__** attribute.

```
import pandas as pd
print(pd.__version__)
```

Output:

'1.4.4'

Introduction to Data Structures

Pandas deals with the following two data structures :-

1. Series

2. DataFrame

These data structures are built on top of Numpy array, which means they are fast.

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

Dimension & Description

The best way to think of these data structures is that the higher dimensional data structure is a container of its lower dimensional data structure.

For example,

DataFrame is a container of Series,

Data Structure	Dimensions	Description
Series	1D	labeled homogeneous array, size immutable.
Data Frames	2D	labeled, size-mutable tabular structure with potentially heterogeneously typed columns.

Building and handling two or more dimensional arrays is a tedious task, burden is placed on the user to consider the orientation of the dataset when writing functions. But using Pandas data structures, the mental effort of the user is reduced.

Mutability

All Pandas data structures are value mutable (can be changed) and except Series all are size mutable. Series is size immutable.

smetechnologies
GOING THE FUTURE WITH AI
Floor, Express Ho... Kochi, Ernakulam, Kerala-682017
Ph. No: +91 9778191...5

Series:

	apples
0	3
1	2
2	0
3	1

Series is a one-dimensional labeled array capable of holding data of any type (integer, string, float, python objects, etc.). The axis labels are collectively called index.

pandas.Series

A pandas Series can be created using the following constructor –

Syntax:

pandas.Series(data, index, dtype, copy)

1. ***data:*** data takes various forms like ndarray, list, constants

2. ***index:*** Index values must be unique and hashable, same length as data. Default np.arrange(n) if no index is passed.

3. ***dtype:*** dtype is for data type. If None, data type will be inferred

4. ***copy:*** Copy data. Default False

A series can be created using various inputs like −

1. List
2. Array

smedtechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7781015215

3. Dict

4. Scalar value or constant

Create an Empty Series

A basic series, which can be created is an Empty Series.

```
#import the pandas library and aliasing as pd
import pandas as pd
s = pd.Series()
print(s)
```

output:

Series([], dtype: float64)

1. Create a Series from list

If data is list, then index passed must be of the same length. If no index is passed, then by default index will be range(n) where n is array length, i.e., [0,1,2,3…. range(len(array))-1].

```
import pandas as pd
import numpy as np
data=[1,2,3,4]
s=pd.Series(data)
print(s)
```

output:

0 1

1 2

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

2 3

3 4

dtype: int64

2. Create a Series from ndarray

If data is an ndarray, then index passed must be of the same length. If no index is passed, then by default index will be range(n) where n is array length, i.e., [0,1,2,3…. range(len(array))-1].

```
#import the pandas library and aliasing as pd
import pandas as pd
import numpy as np
data = np.array(['a','b','c','d'])
s = pd.Series(data)
print(s)
```

output:

0 a

1 b

2 c

3 d

dtype: object

```
#import the pandas library and aliasing as pd
import pandas as pd
import numpy as np
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
data = np.array(['a','b','c','d'])
s = pd.Series(data,index=[100,101,102,103])
print(s)
```

output:

```
100    a
101    b
102    c
103    d
dtype: object
```

3. Create a Series from dict

A dict can be passed as input and if no index is specified, then the dictionary keys are taken in a sorted order to construct index. If index is passed, the values in data corresponding to the labels in the index will be pulled out.

```
#import the pandas library and aliasing as pd
import pandas as pd
import numpy as np
data = {'a' : 0., 'b' : 1., 'c' : 2.}
s = pd.Series(data)
print(s)
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778190515

output:

a   0.0

b   1.0

c   2.0

dtype: float64

Observe − Dictionary keys are used to construct

index.

```
#import the pandas library and aliasing as pd
import pandas as pd
import numpy as np
data = {'a' : 0., 'b' : 1., 'c' : 2.}
s = pd.Series(data,index=['b','c','d','a'])
print(s)
```

output:

b   1.0

c   2.0

d   NaN

a   0.0

dtype: float64

Observe − Index order is persisted and the missing

element is filled

with NaN (Not a Number).

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

4. Create a Series from Scalar

If data is a scalar value, an index must be provided. The value will be repeated to match

the length of index

```
#import the pandas library and aliasing as pd
import pandas as pd
import numpy as np
s = pd.Series(5, index=[0, 1, 2, 3])
print(s)
```

output:

```
0    5

1    5

2    5

3    5

dtype: int64
```

Accessing Data from Series with Position

Data in the series can be accessed similar to that in an ndarray.

```
import pandas as pd
s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])
print(s)
#retrieve the first element
print(s[0])
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

output:

a 1

b 2

c 3

d 4

e 5

dtype: int64

1

Retrieve the first three elements in the Series

```
import pandas as pd
s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])
print(s)
#retrieve the first three element
print(s[:3])
```

output:

a 1

b 2

c 3

d 4

e 5

dtype: int64

a 1

b 2

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
c    3
dtype: int64
```

Retrieve the last three elements

```
import pandas as pd
s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])
print(s)
#retrieve the last three element
print(s[-3:])
```

output:

```
a    1
b    2
c    3
d    4
e    5
dtype: int64
c    3
d    4
e    5
dtype: int64
```

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

Retrieve Data Using Label (Index)

A Series is like a fixed-size dict in that you can get and set values by index label.

```
import pandas as pd
s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])
print(s)
#retrieve a single element
print(s['b'])
```

output:

a 1

b 2

c 3

d 4

e 5

dtype: int64

2

Retrieve multiple elements using a list of index

- **Label values**

```
import pandas as pd
s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])
print(s)
#retrieve multiple elements
```

smectechnologies
GUIDING THE FUTURE WITH AI
7 Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph No: +91 9778191215

```
print(s[['a','c','d']])
```

output:

```
a    1
b    2
c    3
d    4
e    5
dtype: int64
a    1
c    3
d    4
dtype: int64
```

Series Basic Functionality

1. ***axes:*** Returns a list of the row axis labels
2. ***dtype:*** Returns the dtype of the object.
3. ***empty:*** Returns True if series is empty.
4. ***ndim:*** Returns the number of dimensions of the underlying data, by definition 1.
5. ***size:*** Returns the number of elements in the underlying data.
6. ***values:*** Returns the Series as ndarray.
7. ***head():*** Returns the first n rows.

smetechnologies
GUIDING THE FUTURE WITH AI
7 Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph No: +91 9778 4215

8. tail(): Returns the last n rows.

```
import pandas as pd
import numpy as np
#Create a series with 4 random numbers
s = pd.Series(np.random.randn(4))
print(s)
```

output:

```
0    1.292898
1    0.619236
2    0.251232
3    1.252733
dtype: float64
```

1) axes: Returns the list of the labels of the series.

```
import pandas as pd
import numpy as np
#Create a series with 4 random numbers
s = pd.Series(np.random.randn(4))
print ("The axes are:")
print(s.axes)
```

output:

```
The axes are:
[RangeIndex(start=0, stop=4, step=1)]
```

2) empty: Returns the Boolean value saying whether the Object

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 97 8191215

is empty or not.

True indicates that the object is empty.

```
import pandas as pd
import numpy as np
#Create a series with 4 random numbers
s = pd.Series(np.random.randn(4))
print ("Is the Object empty?")
print(s.empty)
```

output:

Is the Object empty?

False

3) ndim: Returns the number of dimensions of the object. By definition, a Series is a 1D data structure, so it returns

```
import pandas as pd
import numpy as np
#Create a series with 4 random numbers
s = pd.Series(np.random.randn(4))
print(s)
print("The dimensions of the object:")
print(s.ndim)
```

output:

0   0.274029

1   -0.648602

2   0.513004

smectechnologies
GUIDING THE FUTURE WITH AI
IT Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Phone: +91 97 8219 7115

```
3    0.674941
dtype: float64
The dimensions of the object:
1
```

4) size: Returns the size (length) of the series.

```
import pandas as pd
import numpy as np
#Create a series with 4 random numbers
s = pd.Series(np.random.randn(4))
print(s)
print ("The size of the object:")
print(s.size)
```

output:

```
0    3.375035
1   -0.267950
2    0.552188
3    1.501215
dtype: float64
The size of the object:
4
```

5)values: Returns the actual data in the series as a numpy array.

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
import pandas as pd
```

```
import numpy as np
import pandas as pd
#Create a series with 4 random numbers
s = pd.Series(np.random.randn(4))
print(s)
arr=s.values
print ("The actual data series is:",arr,'and datatype is:',type(arr))
```

output:

```
0    1.351927
1    1.281453
2    2.086591
3   -0.159584
dtype: float64
The actual data series is: [ 1.35192686  1.28145349  2.08659124
-0.15958373] and datatype is: <class 'numpy.ndarray'>
```

6) head & tail: To view a small sample of a Series or the DataFrame object, use the head() and the tail() methods.

head(): returns the first n rows(observe the index values). The default number of elements to display is five, but you may pass a custom number.

```
import pandas as pd
import numpy as np
#Create a series with 4 random numbers
s = pd.Series(np.random.randn(4))
print("The original series is:")
print(s)
```

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7778102215

```
print ("The first two rows of the data series:")
print(s.head(2))
```

output:

The original series is:

0   0.536673

1   -0.564246

2   -1.690460

3   -0.802960

dtype: float64

The first two rows of the data series:

0   0.536673

1   -0.564246

dtype: float64

tail(): returns the last n rows(observe the index values). The default number of elements to display is five, but you may pass a custom number.

```
import pandas as pd
import numpy as np
#Create a series with 4 random numbers
s = pd.Series(np.random.randn(4))
print ("The original series is:")
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7907819215

```
print(s)
print ("The last two rows of the data series:")
print(s.tail(2))
```

output:

The original series is:

0   -0.211240

1   -0.193382

2   0.018786

3   -1.546757

dtype: float64

The last two rows of the data series:

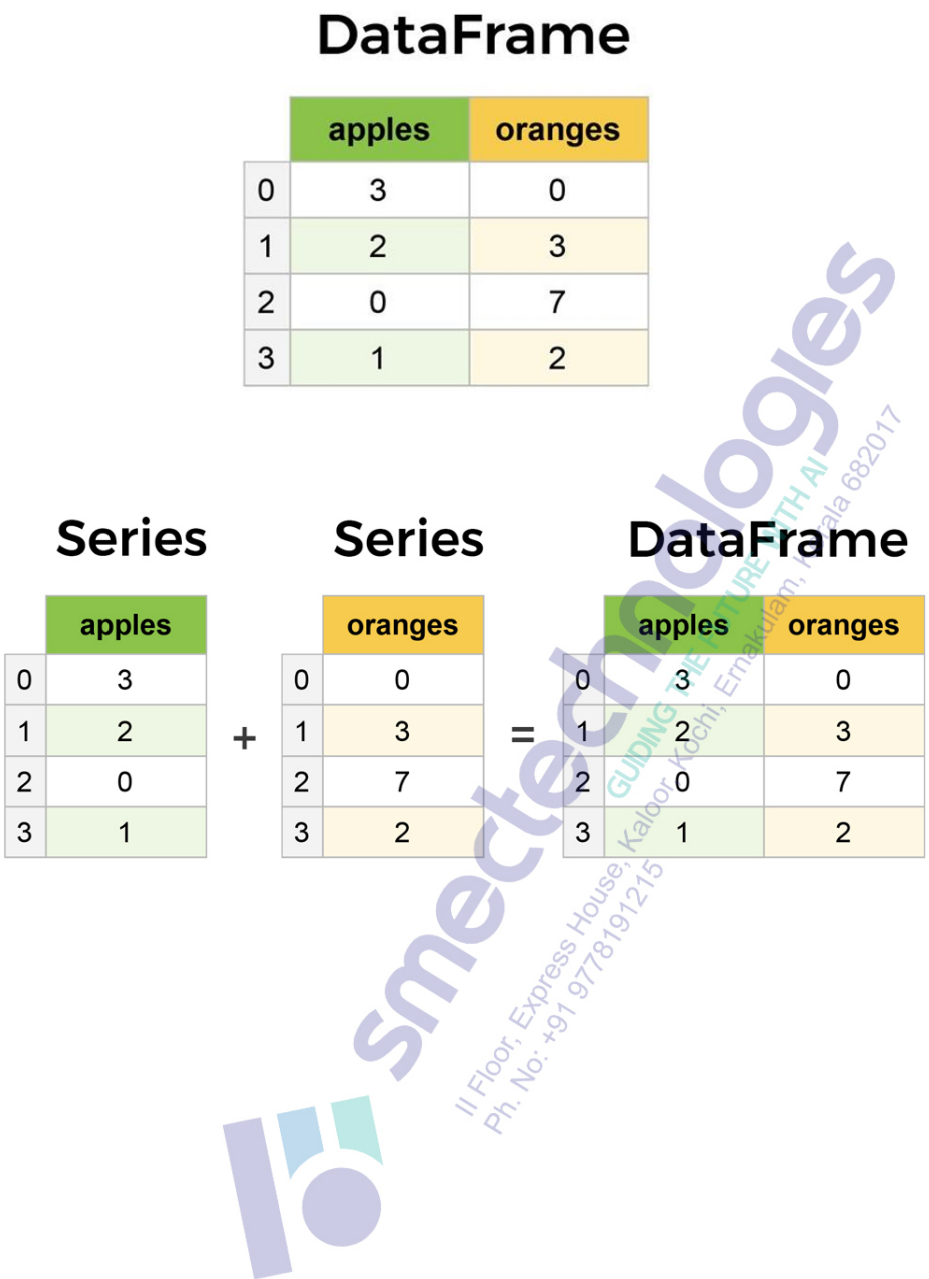
2   0.018786

3   -1.546757

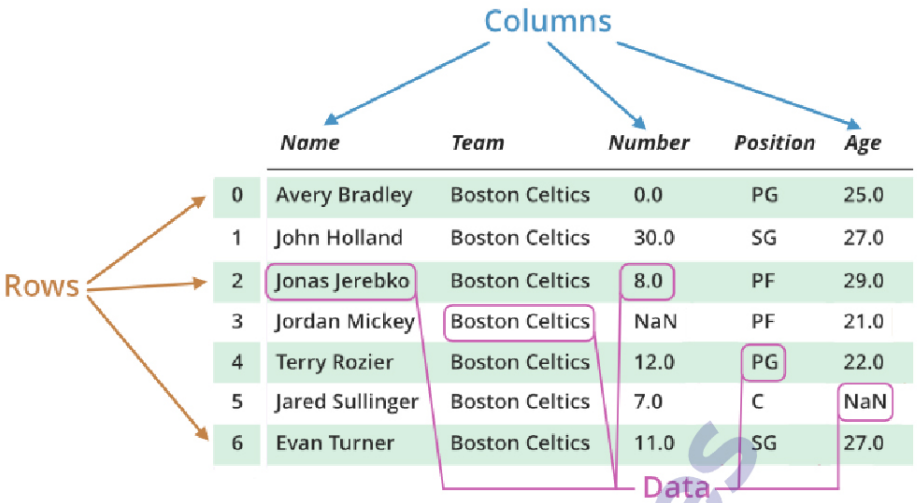
dtype: float64

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

DataFrame



smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215



A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns.

Features of DataFrame

Potentially columns are of different types

Size – Mutable

Labeled axes (rows and columns)

Can Perform Arithmetic operations on rows and columns

pandas.DataFrame

A pandas DataFrame can be created using the following constructor –

Syntax:

```
pandas.DataFrame( data, index, columns, dtype, copy)
```

1 data: data takes various forms like ndarray, series, map, lists, dict, constants and also another DataFrame.

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

2 index: For the row labels, the Index to be used for the resulting frame is Optional Default np.arange(n) if no index is passed.

3 columns: For column labels, the optional default syntax is - np.arange(n). This is only true if no index is passed.

4 dtype: Data type of each column.

5 copy: This command (or whatever it is) is used for copying of data, if the default is False.

A pandas DataFrame can be created using various inputs like −

1. ***Lists***
2. ***dict***
3. ***Series***
4. ***Numpy ndarrays***
5. ***Another DataFrame***

Create an Empty DataFrame

A basic DataFrame, which can be created is an Empty Dataframe

```
#import the pandas library and aliasing as pd
import pandas as pd
df = pd.DataFrame()
print (df)
```

output:

Empty DataFrame

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778901215

Columns: []

Index: []

1. Create a DataFrame from Lists

The DataFrame can be created using a single list or a list of lists.

```
import pandas as pd
data = [1,2,3,4,5]
df = pd.DataFrame(data)
print(df)
```

output:

```
   0
0  1
1  2
2  3
3  4
4  5
```

Create a dataframe with column index

```
import pandas as pd
data = [['Alex',10],['Bob',12],['Clarke',13]]
df = pd.DataFrame(data,columns=['Name','Age'])
print(df)
```

smectechnologies
GUIDING THE FUTURE WITH AI
3rd Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph No: +91 9778191215

output:

```
   Name  Age
0   Alex   10
1    Bob   12
2  Clarke   13
```

Dataframe with column index and data type

```
import pandas as pd
data = [['Alex',10],['Bob',12],['Clarke',13]]
df = pd.DataFrame(data,columns=['Name','Age'])
print(df)
```

output:

```
Name  Age
0   Alex   10
1    Bob   12
2   Clarke   13
```

To identify the datatype of each column in dataframe

```
print(df.dtypes)
```

```
Output:
Name    object
Age      int64
```

smectechnologies
GUIDING THE FUTURE WITH IT
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

dtype: object

To convert a column datatype (typecast)

```
df['Age']=df['Age'].astype('float')
print(df.dtypes)
```

Output:

Name object

Age float64

dtype: object

2. Create a DataFrame from Dict of ndarrays / Lists

All the ndarrays must be of same length. If index is passed, then the length of the index should equal to the length of the arrays.

```
import pandas as pd
data={'Name':['Tom','Jack','Steve','Ricky'],\
      'Age':[28,34,29,42]}
df = pd.DataFrame(data)
print (df)
```

output:

```
   Name  Age
0   Tom   28
1  Jack   34
2 Steve   29
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaithavana, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

3 Ricky 42

Note − Observe the values 0,1,2,3. They are the default index assigned to each using the function range(n).

Let us now create an indexed DataFrame with dictionary and lists.

```
import pandas as pd
Name_list=['Tom','Jack','Steve','Ricky']
Age_list=[28,34,29,42]
data={'Name':Name_list,\
      'Age':Age_list}
df=pd.DataFrame(data,index=['rank1','rank2','rank3','rank4']
)
print (df)
```

Output:

```
        Name  Age
rank1    Tom   28
rank2   Jack   34
rank3  Steve   29
rank4  Ricky   42
```

Note − Observe, the index parameter assigns an index to each row.

Let us now create an indexed DataFrame with dictionary and numpy array.

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
import pandas as pd
Name_arr=np.array(['Tom','Jack','Steve','Ricky'])
Age_arr=np.array([28,34,29,42])
data={'Name':Name_arr,'Age':Age_arr}
df=pd.DataFrame(data,index=['rank1','rank2','rank3','rank4']
)
print (df)
```

Ouput:

Name Age

rank1   Tom   28

rank2   Jack   34

rank3 Steve   29

rank4 Ricky   42

3. Create a DataFrame from List of Dicts

List of Dictionaries can be passed as input data to create a DataFrame.

The dictionary keys are by default taken as column names.

```
import pandas as pd
data = [{'a': 1, 'b': 2},{'a': 5, 'b': 10, 'c': 20}]
df = pd.DataFrame(data)
print(df)
```

output:

smertechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 97 1912

```
   a   b     c
0  1   2   NaN
1  5  10  20.0
```

Note − Observe, NaN (Not a Number) is appended in missing areas.

4. To create a DataFrame by passing a list of dictionaries and the row indices

```
import pandas as pd
data = [{'a': 1, 'b': 2},{'a': 5, 'b': 10, 'c': 20}]
df = pd.DataFrame(data, index=['first', 'second'])
print(df)
```

output:

```
        a   b    c
first   1   2  NaN
second  5  10  20.0
```

5. To create a DataFrame with a list of dictionaries, row indices, and column indices.

```
import pandas as pd
data = [{'a': 1, 'b': 2},{'a': 5, 'b': 10, 'c': 20}]

#With two column indices, values same as dictionary keys
df1 = pd.DataFrame(data, index=['first', 'second'], \
```

smectechnologies
GUIDING THE FUTURE WITH IT
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
                    columns=['a', 'b'])
#With two column indices with one index with other name
df2 = pd.DataFrame(data, index=['first', 'second'], \
                    columns=['a', 'b1'])
print (df1)
print (df2)
```

output:

```
        a   b
first   1   2
second  5   10
        a   b1
first   1 NaN
second  5 NaN
```

Note − Observe, df2 DataFrame is created with a column index other than the dictionary key; thus, appended the NaN's in place. Whereas, df1 is created with column indices same as dictionary keys, so NaN's appended.

6. Create a DataFrame from Dict of Series

Dictionary of Series can be passed to form a DataFrame. The resultant index is the unionof all the series indexes passed.

```
import pandas as pd
Name_Series=pd.Series(['Tom','Jack','Steve'],  \
                      index=['a', 'b', 'c'])
Age_Series=pd.Series([28,34,29,42], \
```

smetechnologies GUIDING THE FUTURE WITH AI Kaithor, Kochi, Ernakulam, Kerala 682017

```
                        index=['a', 'b', 'c', 'd'])
data = { 'Name' : Name_Series, 'Age': Age_Series}
df = pd.DataFrame(data)
print(df)
```

```
  Name  Age
a   Tom   28
b  Jack   34
c Steve   29
d   NaN   42
```

Note – Observe, for the series Name, there is no label 'd' passed, but in the result, for the d label, NaN is appended with NaN.

Column Selection

We will understand this by selecting a column from the DataFrame.

```
import pandas as pd
d = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),
   'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', \
'd'])}
df = pd.DataFrame(d)
print(df)
print (df['one'])
print (df[['one','two']])
```

output:

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Palarivattom, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
   one  two
a  1.0    1
b  2.0    2
c  3.0    3
d  NaN    4
a    1.0
b    2.0
c    3.0
d    NaN
Name: one, dtype: float64
   one  two
a  1.0    1
b  2.0    2
c  3.0    3
d  NaN    4
```

Column Addition

We will understand this by adding a new column to an existing data frame.

```
import pandas as pd
d = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),\
   'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}
df = pd.DataFrame(d)
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
print(df)
# Adding a new column to an existing DataFrame object with column label by passing new series
print ("Adding a new column by passing as Series:")
df['three']=pd.Series([10,20,30],index=['a','b','c'])
print(df)
print ("Adding a new column using the existing columns in DataFrame:")
df['four']=df['one']+df['three']
print (df)
```

output:

```
one  two
a  1.0    1
b  2.0    2
c  3.0    3
d  NaN    4
Adding a new column by passing as Series:
   one  two  three
a  1.0    1   10.0
b  2.0    2   20.0
c  3.0    3   30.0
d  NaN    4    NaN
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7778191215

Adding a new column using the existing columns in DataFrame:

```
   one  two  three  four
a  1.0    1   10.0  11.0
b  2.0    2   20.0  22.0
c  3.0    3   30.0  33.0
d  NaN    4    NaN   NaN
```

Column Deletion

Columns can be deleted or popped; let us take an example to understand it.

```
# Using the previous DataFrame, we will delete a column
# using del function
import pandas as pd
d = {'one' : pd.Series([1, 2, 3],index=['a', 'b', 'c']),\
   'two' : pd.Series([1, 2, 3, 4],index=['a', 'b', 'c', 'd']),\
   'three' : pd.Series([10,20,30], index=['a','b','c'])}
df = pd.DataFrame(d)
print("Our dataframe is:")
print(df)
# using del function
print ("Deleting the first column using DEL function:")
del df['one']
print(df)
# using pop function
print ("Deleting another column using POP function:")
df.pop('two')
```

smetechnologies
CHANGING THE FUTURE WITH AI
II Floor, Express House, Kaloor Kochi Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
print(df)
```

output:

```
Our dataframe is:

   one  two  three
a  1.0    1   10.0
b  2.0    2   20.0
c  3.0    3   30.0
d  NaN    4    NaN
Deleting the first column using DEL function:
   two  three
a    1   10.0
b    2   20.0
c    3   30.0
d    4    NaN
Deleting another column using POP function:
   three
a   10.0
b   20.0
c   30.0
d    NaN
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215